

Complete RTL-to-GDSII Flow Guide Using UART as Example

Introduction

This document explains the complete RTL-to-GDSII ASIC design flow using a UART design as a practical example.

The goal is to demonstrate how a digital hardware design written in Verilog RTL is transformed into a physical silicon layout ready for fabrication.

This guide covers:

- RTL design
- Functional simulation
- Synthesis
- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Routing
- Congestion analysis
- IR-drop analysis
- Final GDSII generation
- Visualization of layouts
- Reports and timing analysis
- Common problems and debugging methods

This workflow uses:

- Verilog HDL
- Yosys
- OpenROAD
- SKY130HD PDK
- KLayout
- GTKWave
- Icarus Verilog

1. Understanding RTL-to-GDSII

RTL-to-GDSII is the process of converting:

RTL Code → Logic Gates → Physical Layout → Fabrication Ready Chip

The entire flow is divided into two major parts:

Stage	Description
Frontend	RTL coding, simulation, synthesis
Backend	Floorplan, placement, CTS, routing, GDSII

2. Project Used in This Guide

Example project:

UART Transmitter + Receiver

Modules:

```
baud_gen.v
uart_tx.v
uart_rx.v
uart_top.v
```

3. Project Directory Structure

Recommended structure:

```
uart-rtl-to-gdsii/
├─ rtl/
├─ testbenches/
├─ simulations/
│   └─ waveforms/
│       └─ images/
├─ synthesis/
│   └─ netlists/
│   └─ reports/
│       └─ schematics/
└─ openroad/
```

```
| | | synthesis/
| | | floorplan/
| | | placement/
| | | cts/
| | | routing/
| | | final/
| | | reports/
| | | visualizations/
| | documentation/
```

4. Writing RTL Code

RTL (Register Transfer Level) describes hardware behavior.

Example UART blocks:

Module	Purpose
baud_gen	Generates baud clock
uart_tx	UART transmitter
uart_rx	UART receiver
uart_top	Top-level integration

Example RTL snippet:

```
module uart_tx(
    input clk,
    input rst_n,
    input tx_start,
    input [7:0] data,
    output reg tx
);
```

5. Writing Testbenches

Testbenches verify RTL functionality.

Example:

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

Simulation verifies:

- Functional correctness
- State transitions
- Timing behavior
- UART transmission
- RX/TX synchronization

6. Running Functional Simulation

Compile using Icarus Verilog:

```
iverilog -o uart_top.out uart_top.v uart_tx.v uart_rx.v baud_gen.v uart_top_tb.v
```

Run:

```
vvp uart_top.out
```

Open waveform:

```
gtkwave uart_top.vcd
```

7. Common Simulation Problems

Error: Unknown module

Cause

Missing RTL file.

Fix

Include all Verilog files during compilation.

Error: No waveform generated

Cause

Missing dump commands.

Fix

Add:

```
$dumpfile("uart_top.vcd");  
$dumpvars(0, uart_top_tb);
```

8. Understanding Synthesis

Synthesis converts RTL into logic gates.

Transformation:

```
always blocks → flip-flops  
if/else → multiplexers  
logic operators → gates
```

Tool used:

```
Yosys
```

9. Running Synthesis

Open Yosys:

```
yosys
```

Run:

```
read_verilog uart_top.v uart_tx.v uart_rx.v baud_gen.v
hierarchy -top uart_top
synth
abc -liberty sky130.lib
write_verilog uart_synth.v
```

Generated:

```
uart_synth.v
```

This is the gate-level netlist.

10. Understanding Gate-Level Netlist

The synthesized design contains:

- AND gates
- OR gates
- NAND gates
- NOR gates
- XOR gates
- Flip-flops
- Buffers
- Multiplexers

Example:

```
sky130_fd_sc_hd__nand2_1
sky130_fd_sc_hd__dfxtp_1
```

These are standard cells from SKY130.

11. Generating Gate-Level Schematic

Inside Yosys:

```
show -format png -prefix uart_gatelevel uart_top
```

Generated:

```
uart_gatelevel.png
```

This visualizes the synthesized logic network.

12. Common Yosys Error

Error

```
ERROR: For formats different than 'ps' or 'dot' only one module must be selected.
```

Cause

Top module not specified.

Fix

Run:

```
hierarchy -top uart_top
```

Then:

```
show -format png -prefix uart_gatelevel uart_top
```

13. Understanding Physical Design

After synthesis:

```
Logic gates exist only logically.
```

Physical design converts them into:

- Standard cell placement
 - Metal routing
 - Power network
 - Clock trees
 - Final silicon geometry
-

14. Setting Up OpenROAD Flow

Create config file:

```
config.mk
```

Example:

```
export DESIGN_NAME = uart_top  
export PLATFORM = sky130hd
```

Add RTL files and SDC constraints.

15. Understanding SDC Constraints

SDC defines timing constraints.

Example:

```
create_clock [get_ports clk] -period 20
```

This tells tools:

```
Clock period = 20ns
```

16. Running RTL-to-GDSII Flow

Go to flow directory:


```
cd OpenROAD-flow-scripts/flow
```

Run:

```
make DESIGN_CONFIG=./designs/sky130hd/uart/config.mk
```

This launches the full backend flow.

17. Backend Flow Stages

Stage 1: Synthesis

Generated files:

```
1_2_yosys.v  
1_synth.odt
```

Purpose:

- Gate mapping
 - Area optimization
 - Timing optimization
-

Stage 2: Floorplanning

Generated files:

```
2_floorplan.odt
```

Purpose:

- Define chip dimensions
- Create core area
- Reserve routing space
- Add power planning

Visualization shows:

- Chip outline
 - IO pin locations
 - Core region
-

Stage 3: Placement

Generated files:

3_place.odb

Purpose:

- Place all standard cells
- Reduce wirelength
- Improve timing

Visualization shows:

- Standard cell rows
 - Cell distribution
 - Density
-

Stage 4: CTS (Clock Tree Synthesis)

Generated files:

4_cts.odb

Purpose:

- Distribute clock evenly
- Reduce clock skew
- Insert buffers

Visualization shows:

- Clock routing
 - Clock buffers
 - Clock branches
-

Stage 5: Routing

Generated files:

5_route.odb

Purpose:

- Connect all nets
- Add metal layers
- Add vias

Visualization shows:

- Metal routing
 - Signal paths
 - Vias
 - Interconnect network
-

Stage 6: Fill and Finishing

Generated files:

6_final.gds

Purpose:

- Add fill cells
 - Final cleanup
 - Generate fabrication layout
-

18. Understanding GDSII

GDSII is the final layout database.

Contains:

- Transistor geometries
- Metal layers
- Contacts
- Vias

- Power rails
- Final routing

This is the file sent for fabrication.

19. Viewing Layouts Using KLayout

Install:

```
sudo apt install klayout -y
```

Open final GDS:

```
klayout 6_final.gds
```

20. Viewing Different Layout Stages

You can visualize:

Stage	File
Synthesis	1_synth.odb
Floorplan	2_floorplan.odb
Placement	3_place.odb
CTS	4_cts.odb
Routing	5_route.odb
Final	6_final.gds

21. Understanding ODB Files

ODB files contain:

- Placement database
- Routing database
- Timing data

- Physical information

OpenROAD internally uses these databases.

22. Common KLayout Issue

Problem

Blank layout window.

Cause

Zoom level incorrect.

Fix

Press:

Shift + F

This fits design to screen.

23. Another Common KLayout Issue

Error

Macro not found in LEF file

Cause

LEF technology not loaded.

Fix

Usually safe to ignore for quick viewing.

Better approach:

Open:

```
6_final.gds
```

instead of DEF.

24. OpenROAD GUI Error

Error

```
qt.qpa.xcb: could not connect to display
```

Cause

GUI display server missing.

Fix

1. Start XLaunch/VcXsrv
2. Export DISPLAY variable

```
export DISPLAY=$(grep nameserver /etc/resolv.conf | awk '{print $2}'):0
```

25. Docker Permission Problem

Error

```
dmesg: read kernel buffer failed: Operation not permitted
```

Cause

WSL container restrictions.

Fix

Usually safe to ignore.

Not related to flow failure.

26. CTS Illegal Instruction Error

Error

```
child killed: illegal instruction
```

Cause

Possible reasons:

- WSL instability
- Docker CPU issue
- Unsupported instruction set
- OpenROAD binary mismatch

Fixes

1. Restart WSL

```
wsl --shutdown
```

1. Rebuild OpenROAD
2. Retry flow
3. Reduce thread count
4. Update Docker

27. Understanding Congestion Maps

Congestion maps show:

```
Which areas are difficult to route
```

Colors:

Color	Meaning
Blue	Low congestion
Green	Medium
Red	High congestion

28. Understanding IR Drop Analysis

IR-drop analysis measures:

Voltage drop across power network

Important for:

- Power integrity
- Reliable operation
- Timing stability

29. Understanding Timing Reports

Important reports:

Report	Purpose
synth_stat.txt	Synthesis stats
3_global_place.rpt	Placement report
4_cts_final.rpt	CTS report
5_route_drc.rpt	Routing DRC report
6_finish.rpt	Final report

30. Understanding Final Layout

The final GDSII contains:

- Thousands of polygons
- Multiple metal layers
- Standard cells
- Clock networks
- Power rails
- Signal routing

This is an actual silicon representation.

31. Files Worth Saving for GitHub

Important files:

```
RTL files
Testbenches
Waveforms
Gate-level netlists
Reports
Final GDSII
Screenshots
Routing images
Congestion maps
IR-drop analysis
```

32. Recommended GitHub Organization

```
openroad/
├─ synthesis/
├─ floorplan/
├─ placement/
├─ cts/
├─ routing/
├─ final/
├─ reports/
└─ visualizations/
```

33. What Was Achieved in This Flow

Using the UART example, the following were completed:

- RTL development
- Simulation
- Synthesis
- Standard cell mapping
- Floorplanning
- Placement optimization
- Clock tree generation
- Detailed routing

- Congestion analysis
 - IR-drop analysis
 - Final GDSII generation
-

34. Industrial Relevance

This flow represents the same core backend ASIC methodology used in:

- Semiconductor companies
 - ASIC startups
 - Processor design teams
 - SoC development
 - AI accelerator design
 - FPGA-to-ASIC migration flows
-

35. Future Improvements

Possible extensions:

- Multi-clock designs
 - SRAM integration
 - DRC verification
 - LVS verification
 - Timing closure optimization
 - Power optimization
 - Antenna fixing
 - Scan chain insertion
 - Formal verification
 - Tapeout preparation
-

36. Final Conclusion

RTL-to-GDSII is the complete transformation of a digital design into a manufacturable silicon layout.

Using open-source tools such as:

- Yosys
- OpenROAD
- SKY130
- KLayout

it is possible to perform an industrial-style ASIC backend flow completely using open-source infrastructure.

The UART example demonstrates the entire journey:

Verilog RTL → Logic Gates → Placement → Routing → Final Silicon Layout

This process provides practical exposure to:

- Digital frontend design
- ASIC backend design
- Physical design concepts
- Timing optimization
- Routing challenges
- Power integrity analysis
- Fabrication-ready layout generation